

Faculty of Engineering and Technology

School of Computer Science and Engineering

B. Tech (Computer Science and Engineering)

COURSE: Operating System

Virtual Memory: Demand Paging – Simulation Activity Report

Submitted to

Ms.Pooja Das

Department of Data Analytics and Mathematical Sciences,
Faculty of Engineering & Technology,
Jain (Deemed-to-be University)

Submitted by

USN	Name
25BTRCN245	Ritika Mahato
25BTRCN240	Prachi Pandit
25BTLCN002	Debarshita sen
25BTLCN003	K. Naga Lakshmi Chandana
25BTLCN004	P. Chanukya Reddy

Date of Submission: 02/11/2026

1. Abstract

This report presents an interactive, browser-based simulation of **demand paging**, the virtual memory technique in which an operating system loads a program's pages into main memory only when they are referenced. The simulator walks through the eight stages of the mechanism: program execution, page table creation, page fault handling, page fetch, table update, program resumption, page replacement and page cleanup. At each step it shows the state of the CPU, page table (valid/invalid bits), main memory frames and backing store. The report explains each step with screenshots, analyses the observed behaviour and the cost of page faults using effective access time, and compares the FIFO, LRU and Optimal replacement algorithms on sample reference strings, including Belady's anomaly. It also discusses the limitations of the simulator and possible improvements, and presents a supplementary experimental study comparing native, virtual machine and container execution on Apple Silicon. The complete source code and all related project work are available on GitHub: https://github.com/Ritika-rli/VM-VS-Container

Keywords: *virtual memory, demand paging, page fault, page replacement, FIFO, LRU, simulation*

2. Introduction

2.1 Virtual memory

Virtual memory lets a program run even when it is larger than the physical memory (RAM) available to it. Each process sees a large, contiguous *logical* address space, while the OS keeps only a part of it in RAM and the rest on disk (the *backing store*). This gives three main benefits: programs larger than RAM can run, more processes can be kept in memory at once (higher multiprogramming), and less I/O is wasted loading code that is never used.

2.2 Paging

In paging, the logical address space is divided into fixed-size blocks called **pages**, and physical memory is divided into blocks of the same size called **frames**. A **page table** maps each logical page number to a physical frame number, so a page can sit in any free frame.

2.3 Demand paging

Demand paging is a "lazy loading" strategy: a page is brought into memory **only when the program actually references it**. A *valid/invalid bit* in every page table entry tells the hardware whether the page is currently in RAM:

- **Valid (v):** the page is in memory and the frame number is usable.
- **Invalid (i):** the page is not in memory (it is on disk), or the address is illegal. Touching it raises a **page fault**.

3. Tools and Setup

Item	Details
Simulation	`virtual_memory_paging.html`, a self-built browser-based demand paging visualizer
Technologies	HTML, CSS (Tailwind via CDN), JavaScript (no external logic libraries)
Run method	Open the file in any modern browser (internet needed once for the Tailwind CDN)
Process modelled	Process P1 with 5 logical pages (Page 0 to Page 4)
Physical memory	4 frames (Frame 0 to Frame 3)
Backing store	Disk holding 8 pages (Page 0 to Page 7)
Controls	**Previous** and **Next Step** buttons, step counter and progress bar

4. Design of the Simulator

The screen is split into a visualization area and a control panel.

- **CPU box:** shows the current CPU/OS activity (for example "Requesting Page 0..." or "Process Terminated").
- **Page Table box:** one row per logical page with three columns: *Logical Page*, *Physical Frame*, *Valid/Invalid*. Valid entries show a green badge, invalid a red badge, and unused entries a grey dash.
- **Main Memory (RAM) box:** four frame slots. Empty frames are grey; occupied frames are blue and show the page they hold.
- **Disk Storage box:** eight slots representing the backing store.
- **Control panel:** shows "Step N of 8", the step title, a plain-English explanation and a progress bar.

The program is data-driven: an array called `stepsData` holds, for each of the 8 steps, the title, description, CPU text, page table rows, RAM contents, disk contents and which components to highlight. A single `renderStep()` function redraws the whole screen from that data, which keeps the simulation simple and reliable.

Colour code used for highlighting:

Glow	Meaning	Used in step
Blue	Component currently active	1, 2, 4, 8

Glow	Meaning	Used in step
Red (with shake)	Page fault	3
Green	Success or update completed	5, 6
Yellow	Page replacement / swap	7

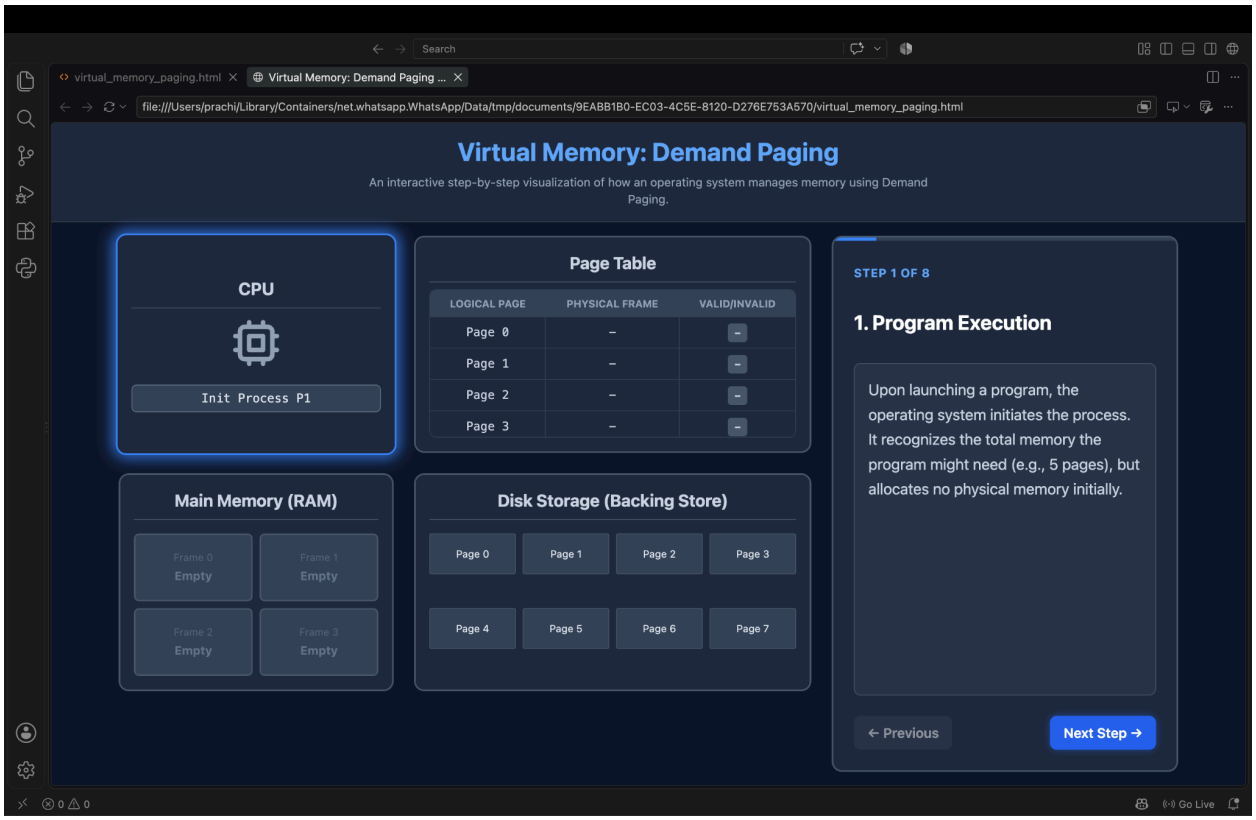
5. Simulation Execution: The Eight Steps

The simulator walks through one complete life cycle of a process under demand paging. The state of each component after every step is summarised below.

Step 1: Program Execution

What happens: When a program is launched, the OS creates a process (P1) and decides how much memory it may use. In demand paging, no pages are loaded yet.

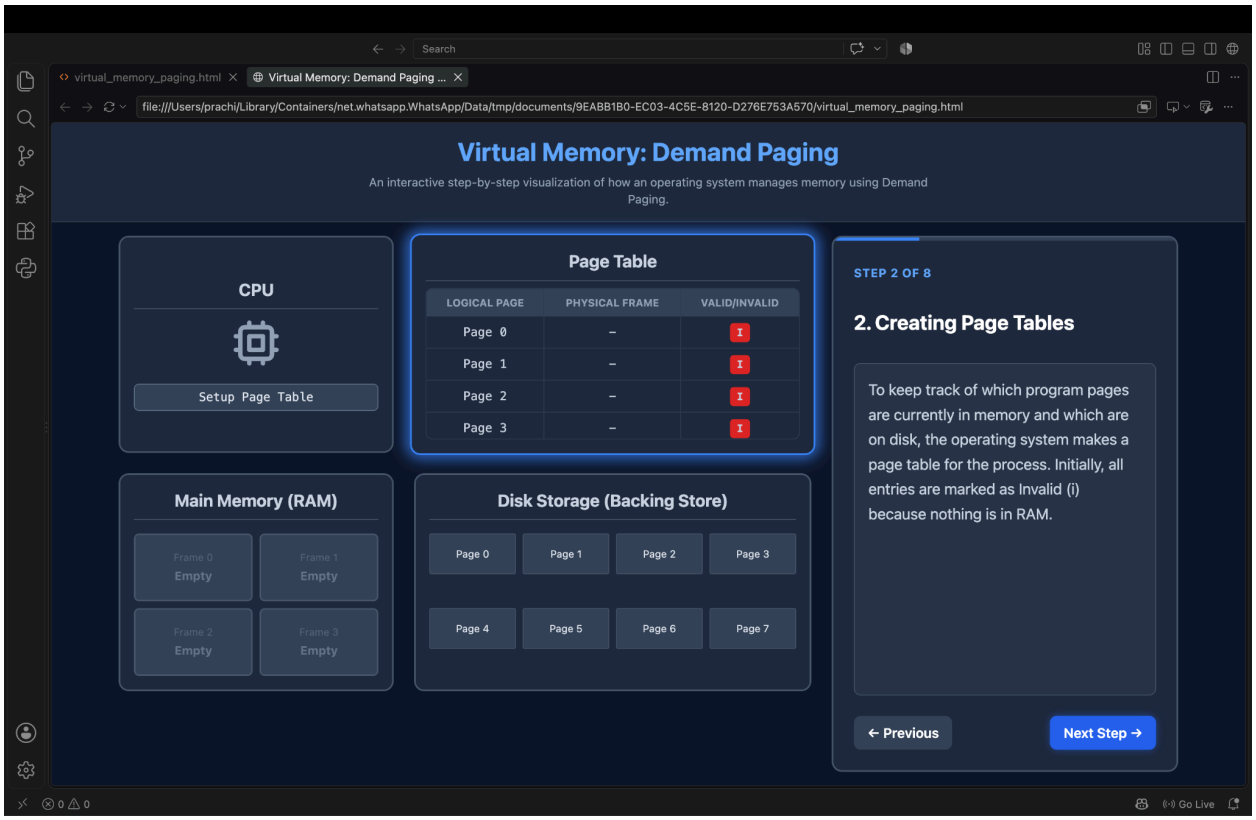
On screen: CPU shows "Init Process P1" (blue glow). The page table rows are all "-", RAM is entirely empty, and all 8 pages are on disk.



Step 2: Creating Page Tables

What happens: The OS builds a page table for the process to track which pages are in RAM and which are on disk. Because nothing is loaded, every entry starts as **Invalid (i)**.

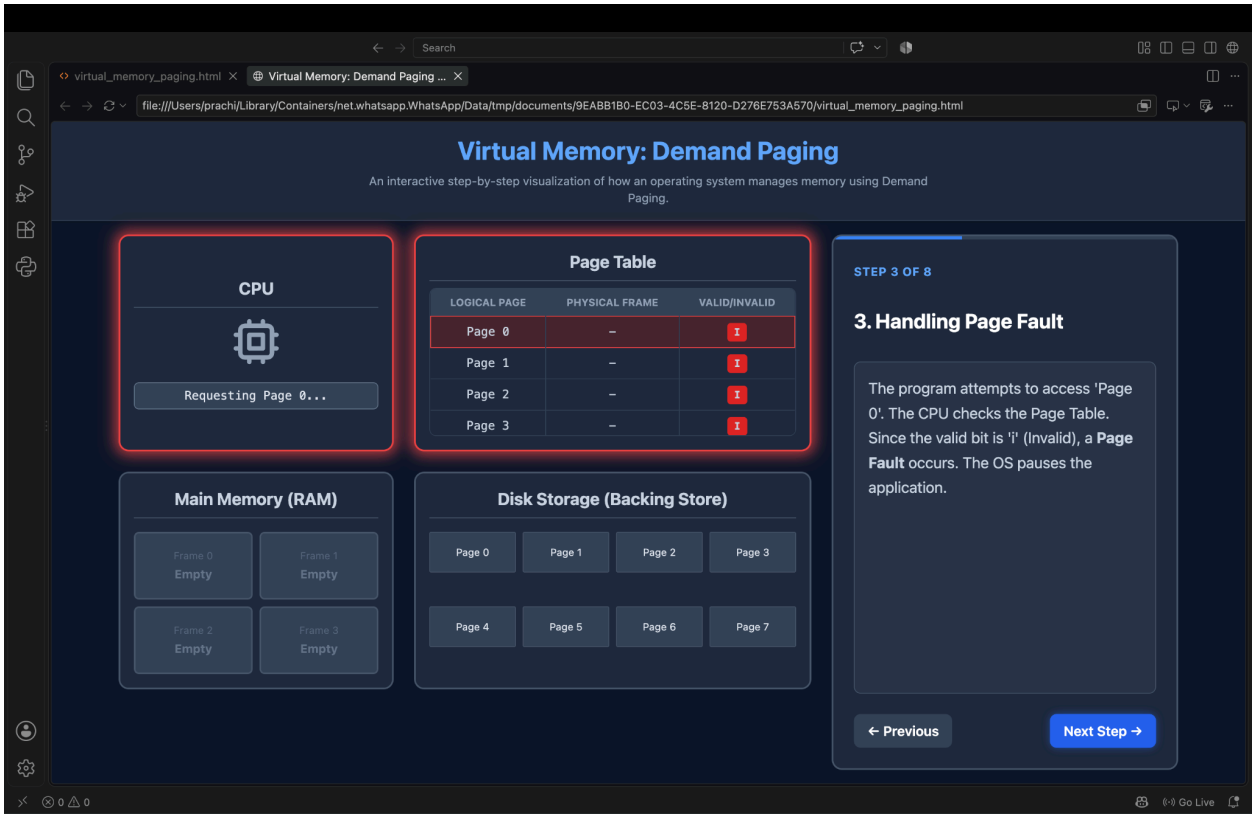
On screen: Page table shows Pages 0 to 4 with frame "-" and valid bit **I** (red). RAM is still empty.



Step 3: Handling Page Fault

What happens: The program tries to access Page 0. The hardware looks up the page table, finds the valid bit is "i", and raises a **page fault** (a trap to the OS). The OS suspends the process and checks that the reference is legal and that the page is on disk.

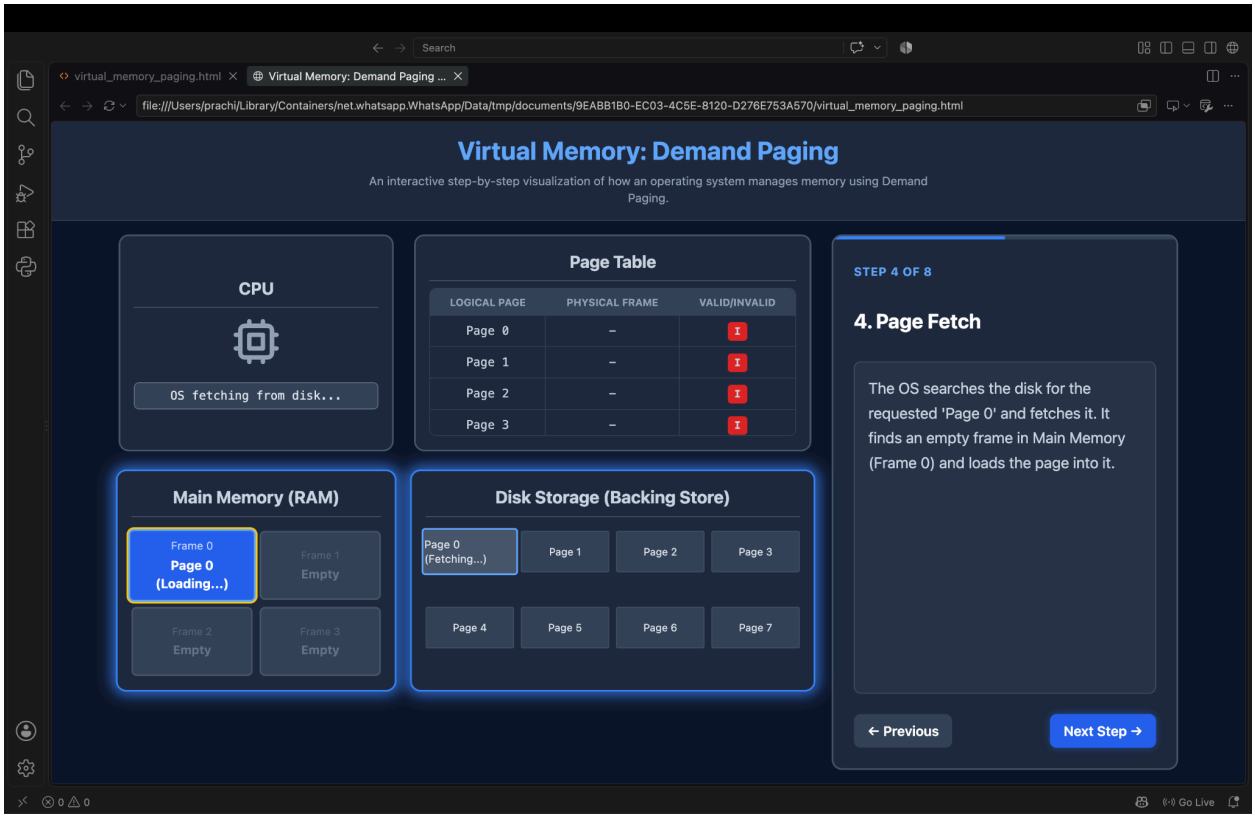
On screen: CPU shows "Requesting Page 0...". The CPU and page table glow red and shake; the Page 0 row is highlighted red.



Step 4: Page Fetch

What happens: The OS locates Page 0 on the backing store, finds a free frame (Frame 0) and schedules a disk read to copy the page into it. This is the slowest part of the whole sequence because it involves disk I/O.

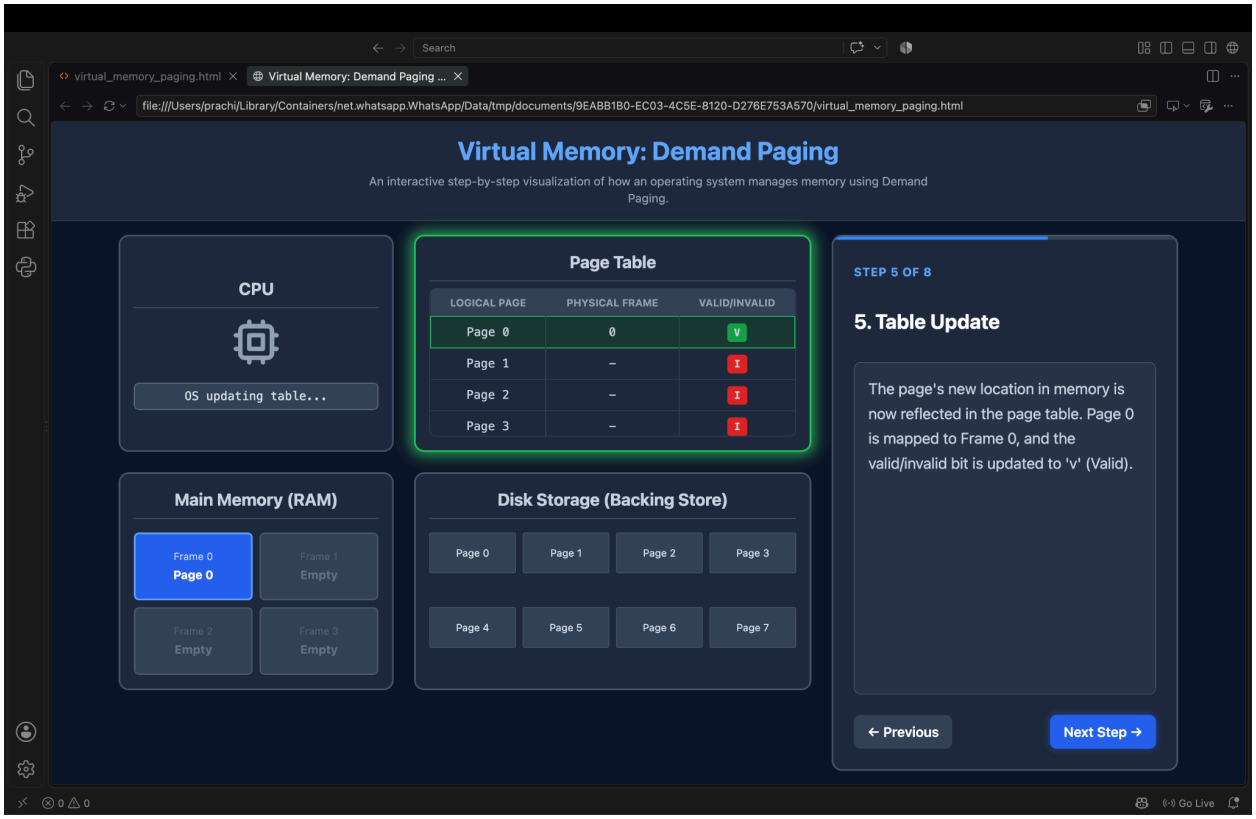
On screen: Disk slot for Page 0 shows "Fetching..." and Frame 0 of RAM shows "Page 0 (Loading...)". The disk and RAM boxes glow blue.



Step 5: Table Update

What happens: After the page is in memory, the OS records its location in the page table: Page 0 maps to Frame 0 and the valid bit changes from **i** to **v**.

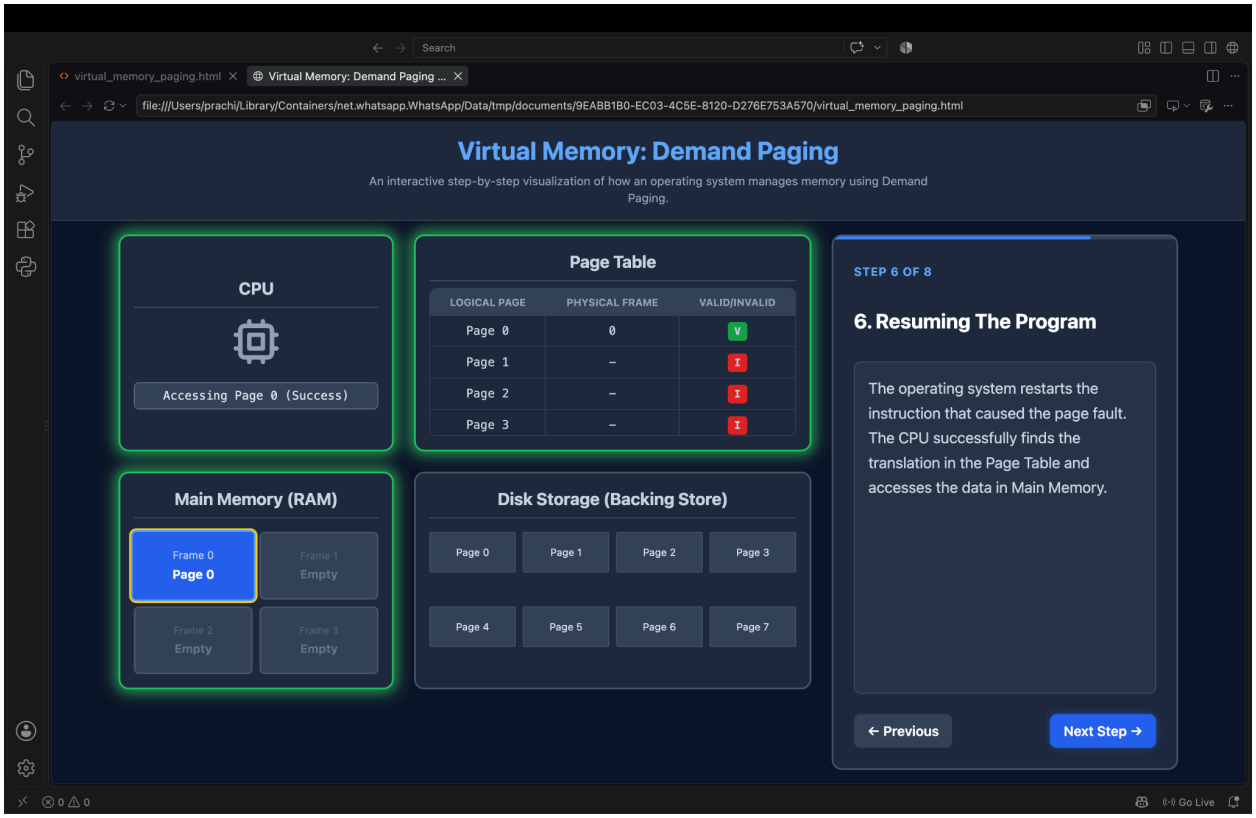
On screen: Row for Page 0 becomes "Frame 0, V" with a green highlight; Frame 0 in RAM shows "Page 0".



Step 6: Resuming the Program

What happens: The OS restarts the instruction that caused the fault. This time the address translation succeeds (page table gives Frame 0) and the data is read from main memory. The program continues as though no fault had occurred.

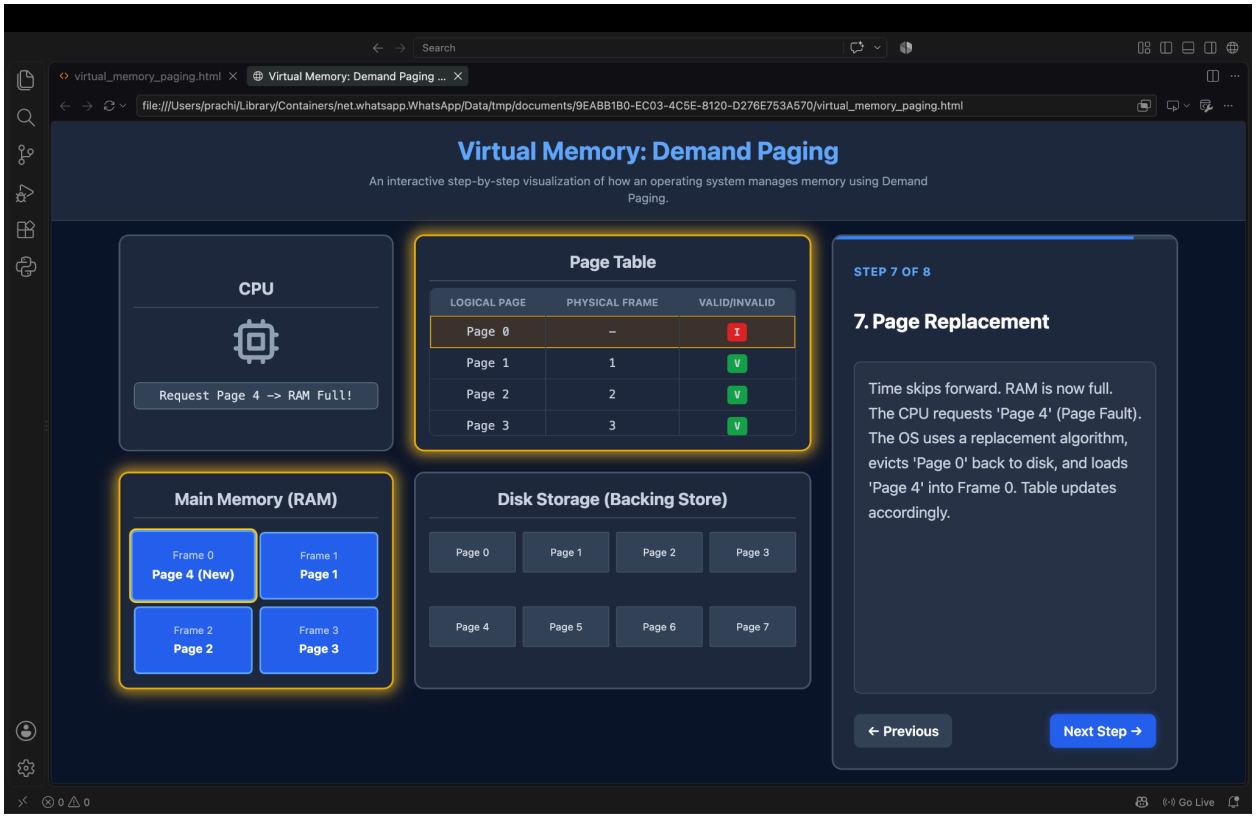
On screen: CPU shows "Accessing Page 0 (Success)"; CPU, page table and RAM glow green.



Step 7: Page Replacement

What happens: After some time all four frames are occupied (Pages 0 to 3). The CPU now requests Page 4, which is not in memory, so another page fault occurs and there is no free frame. The OS runs a **page replacement algorithm** to choose a victim. Here Page 0 is evicted, its page table entry becomes invalid again, and Page 4 is loaded into the freed Frame 0.

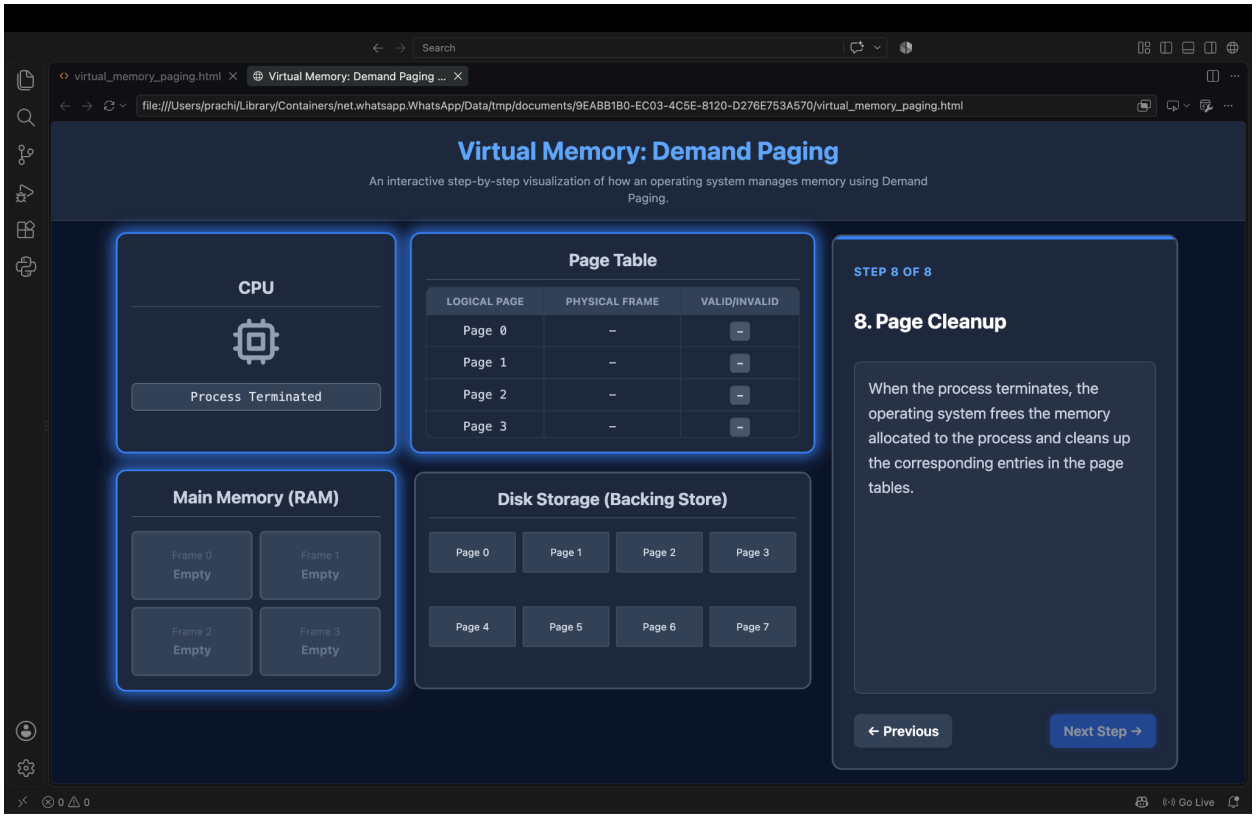
On screen: CPU shows "Request Page 4 -> RAM Full!". Page 0 row turns yellow (now invalid, frame "-"); Page 4 row turns green (Frame 0, valid). RAM shows Page 4, Page 1, Page 2, Page 3 in Frames 0 to 3.



Step 8: Page Cleanup

What happens: When the process terminates, the OS releases all frames it was using and removes its page table entries so the memory can be reused by other processes.

On screen: CPU shows "Process Terminated"; page table rows reset to "-" and RAM is empty again.



Summary of component states

Step	CPU status	Page table (valid pages)	RAM contents
1. Program Execution	Init Process P1	none (all "-")	empty
2. Creating Page Tables	Setup Page Table	none (all invalid)	empty
3. Handling Page Fault	Requesting Page 0	none; Page 0 fault	empty
4. Page Fetch	OS fetching from disk	none yet	Page 0 loading in F0
5. Table Update	OS updating table	Page 0 → F0	Page 0
6. Resuming	Accessing Page 0 (success)	Page 0 → F0	Page 0
7. Page Replacement	Request Page 4, RAM full	Pages 1, 2, 3 → F1, F2, F3; Page 4 → F0; Page 0 invalid	Page 4, 1, 2, 3

Step	CPU status	Page table (valid pages)	RAM contents
8. Page Cleanup	Process terminated	none	empty

6. Observations and Data Analysis

1. **Nothing is loaded at start-up.** After Steps 1 and 2 RAM is empty, so a program begins executing without any of its pages in memory. Memory is used only on demand.
2. **The valid/invalid bit drives everything.** The page fault in Step 3 is triggered purely by the "i" bit, and the change to "v" in Step 5 is what lets the retried instruction succeed in Step 6.
3. **The first reference to any page always faults.** Page 0 faulted the first time it was used (a *cold* or *compulsory* miss) and succeeded the second time.
4. **The page fault is the costly part.** Steps 3 to 5 (trap, disk read, table update) all happen between one failed access and one successful access. A normal memory access is a single step (Step 6).
5. **Page replacement changes two table entries at once.** In Step 7 the victim's entry (Page 0) goes valid → invalid and the new page's entry (Page 4) goes invalid → valid, both pointing at the same Frame 0. The frame is reused; only the mapping changes.
6. **Cleanup returns everything.** Step 8 leaves RAM and the page table in the same state as at the beginning, so the frames are fully reusable.
7. **The process is 5 pages but RAM has 4 frames.** This is exactly the situation virtual memory exists for: the process's logical space (5 pages) is larger than the memory it can hold (4 frames).

6.1 Effect of page faults on performance

The performance of demand paging is measured by the **Effective Access Time (EAT)**:

EAT = (1 − p) × memory access time + p × page-fault service time

where p is the page-fault rate ($0 \le p \le 1$). Using typical textbook values (memory access = 200 ns, page-fault service time = 8 ms = 8,000,000 ns):

Page-fault rate p	EAT (ns)	Slowdown vs 200 ns
0	200	1×
1 in 1,000,000	≈ 208	1.04×
1 in 100,000	≈ 280	1.4×
1 in 1,000	≈ 8,200	≈ 41×

Page-fault rate p	EAT (ns)	Slowdown vs 200 ns
1 in 100	≈ 80,200	≈ 401×

Interpretation: even a one-in-a-thousand fault rate makes memory about 41 times slower. To keep slowdown under 10% (EAT below 220 ns), p must be below about 2.5 in 1,000,000, i.e. fewer than one fault per 400,000 accesses. This is why the choice of replacement algorithm (Section 7) matters so much.

7. Comparative Analysis: FIFO vs LRU vs Optimal

The simulator shows *that* a replacement happens in Step 7, but not *which* algorithm picked the victim. To compare algorithms, the same page-reference string was run through three policies (computed with a short Python script following the standard definitions):

- **FIFO:** replace the page that has been in memory the longest.
- **LRU (Least Recently Used):** replace the page not used for the longest time.
- **Optimal (OPT):** replace the page that will not be used for the longest time in the future. It cannot be implemented in practice and is used as a benchmark.

Reference string A: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 (20 references)

Frames	FIFO faults	LRU faults	OPT faults	FIFO fault rate	LRU fault rate	OPT fault rate
3	15	12	9	75%	60%	45%
4	10	8	8	50%	40%	40%

Reference string B (Belady's anomaly string): 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (12 references)

Frames	FIFO faults	LRU faults	OPT faults
3	9	10	7
4	10	8	6

Findings:

- On string A, LRU produced 3 fewer faults than FIFO with 3 frames (12 vs 15) and 2 fewer with 4 frames (8 vs 10), because it keeps recently used pages that are likely to be used again (locality of reference).
- OPT is the lower bound: no algorithm can do better than 9 faults (3 frames) or 8 faults (4 frames) on string A. LRU came close, FIFO was furthest.
- On string B, FIFO shows **Belady's anomaly**: giving it *more* memory (3 → 4 frames) made it *worse* (9 → 10 faults). LRU and OPT never show this (they are *stack algorithms*), and both improved with more frames.

- Note that LRU is not always better than FIFO for a given string: with 3 frames on string B, FIFO (9) beat LRU (10). LRU's advantage is on programs with good locality.

Criterion	FIFO	LRU	OPT
Idea	Oldest page leaves	Least recently used leaves	Used furthest in future leaves
Implementation	Very simple (queue)	Needs counters or a stack/hardware support	Impossible (needs the future)
Fault performance	Weakest	Good	Best (benchmark)
Belady's anomaly	Yes	No	No

8. Technical Discussion and Reflection

8.1 What the simulation shows well

- It separates the logical view (page table) from the physical view (RAM and disk) so the role of translation is clear.
- Colour and animation link each step to a specific component, which makes the fault-handling sequence easy to follow.
- Stepping forward and backward lets a learner replay the critical Steps 3 to 6 as often as needed.

8.2 Limitations

1. **Fixed script, not a live simulation.** The 8 states are pre-defined. The user cannot enter their own reference string or change the number of frames.
2. **Replacement algorithm is not shown.** Step 7 evicts Page 0 without computing a victim. FIFO would pick Page 0 here (it was loaded first), but LRU might not, since Page 0 could have been used recently.
3. **A time jump hides several faults.** Between Steps 6 and 7 the simulator "skips forward" so Pages 1 to 3 appear in RAM without showing their faults.
4. **No timing or statistics.** Page-fault count, hit ratio and effective access time are not computed or displayed.
5. **Dirty bit not modelled.** In a real OS, an evicted page is written back to disk only if it was modified. The disk display does not change when Page 0 is evicted.
6. **Small and single-process.** One process, 4 frames and no TLB, multi-level page table, or thrashing.
7. **Minor inconsistency.** Step 1 says the program may need 5 pages while the disk panel shows 8 pages.

8.3 Practical implications

- Demand paging works well only because of **locality of reference**: programs tend to reuse a small working set of pages, so faults are rare after warm-up.
- If a process has too few frames, it faults constantly and the system spends more time swapping than computing. This is **thrashing**, and operating systems avoid it by using working-set or page-fault-frequency policies.
- Real systems add a **TLB** to speed up translation, use **copy-on-write** and **pre-paging**, and approximate LRU (clock algorithm, second-chance) because exact LRU is too expensive in hardware.

8.4 Possible improvements

- Add input boxes for the reference string and number of frames.
- Add a selector for FIFO, LRU and OPT and show a fault/hit table that updates live.
- Show a running fault counter, hit ratio and EAT, and a bar chart comparing algorithms.
- Add a dirty-bit option and a TLB to show more realistic behaviour.

9. Conclusion

The simulator clearly demonstrates the eight stages of demand paging: process start, page table creation, page fault, page fetch, table update, program resumption, page replacement and cleanup. It shows that pages are loaded only when referenced, that the valid/invalid bit controls fault detection, and that a page fault is far more expensive than a normal memory access, so the fault rate must be kept extremely low. The comparative analysis showed that LRU generally produces fewer page faults than FIFO (12 vs 15 faults on the sample string with 3 frames) and approaches the optimal algorithm, while FIFO can suffer from Belady's anomaly. Overall, the activity linked the theory of virtual memory to observable system behaviour and showed why page replacement policy has a direct impact on performance.

10. References

1. A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, Wiley: chapter on Virtual Memory.
2. W. Stallings, *Operating Systems: Internals and Design Principles*, Pearson.
3. A. S. Tanenbaum, H. Bos, *Modern Operating Systems*, Pearson.
4. Course lecture slides: *Virtual memory: Demand Paging*.
5. Simulation source file: `virtual_memory_paging.html`.